

Capa de datos distribuida y pipeline analítico paralelo

Sistema de Gestión de Solicitudes de Soporte Técnico de Internet

Entrega 3 — Aplicaciones Distribuidas (ISR-701)

Carlos José Carpio Mendoza

Cristhian Daniel Pacheco Cárdenas

Robinson Rodrigo Cando Moreno

Jeremy Álvarez

Equipo ACC

Facultad de Ciencias de la Computación

Universidad Técnica Estatal de Quevedo

Julio 2026

Resumen

Objetivo. Consolidar la capa de datos distribuida y el pipeline analítico paralelo del Sistema de Soporte Técnico ISP (equipo ACC), evaluando su rendimiento y tolerancia a fallos. **Método.** Se fragmentó `tickets` por rango de `fecha_apertura` sobre un clúster de tres nodos de CockroachDB, verificando completitud, reconstrucción y disyunción [1]; se instrumentó con métricas Prometheus validadas con `promtool` y pruebas Testcontainers contra `SERIALIZABLE` real; y se rediseñó un pipeline de Spark sobre 600,000 incidencias con cinco transformaciones, contrastado mediante protocolo experimental (Jain [2], Wohlin et al. [3]) frente a un baseline en pandas. **Resultados.** El clúster aplica correctamente aislamiento serializable ante escrituras concurrentes conflictivas (SQLSTATE 40001), las tres métricas pasan el `linter` de Prometheus sin advertencias, y el pipeline detecta 380,753 incidencias reinincidentes (68.3 %) sobre 557,701 filas filtradas. El protocolo estadístico completo (50 corridas) muestra que, para este volumen de datos, el baseline secuencial en pandas fue significativamente más rápido que Spark incluso con ocho núcleos ($p \approx 5 \times 10^{-10}$), aunque Spark sí exhibe un *speedup* interno real al escalar de uno a ocho núcleos (p de Amdahl $\approx 0,53$). La tolerancia a fallos permanece pendiente. **Conclusión.** La fragmentación y la instrumentación aplicadas son correctas y verificables; el protocolo revela además que el paralelismo distribuido no es gratuito y solo se justifica a partir de cierta escala de datos.

Abstract

Resumen

Objective. To consolidate the distributed data layer and the parallel analytics pipeline of the ISP Technical Support Ticket Management System (team ACC), evaluating their performance and fault tolerance. **Method.** The `tickets` table was fragmented by range on `fecha_apertura` over a three-node CockroachDB cluster, verifying completeness, reconstruction and disjointness [1]; the service was instrumented with Prometheus metrics validated with `promtool` and Testcontainers tests against real `SERIALIZABLE` isolation; and a Spark pipeline was redesigned over 600,000 incidents with five transformations, contrasted via an experimental protocol against a pandas baseline. **Results.** The cluster correctly enforces serializable isolation under conflicting concurrent writes (SQLSTATE 40001), all three required metrics pass the Prometheus linter without warnings, and the pipeline detects 380,753 recurring incidents (68.3 %) out of 557,701 filtered rows. The full statistical protocol (50 trials) shows that, at this data volume, the sequential pandas baseline was significantly faster than Spark even at eight cores ($p \approx 5 \times 10^{-10}$),

although Spark does show a real internal speedup when scaling from one to eight cores (Amdahl $p \approx 0,53$). Fault-tolerance characterization remains pending. **Conclusion.** The fragmentation and instrumentation applied are correct and empirically verifiable; the protocol further reveals that distributed parallelism is not free and only pays off past a certain data scale.

Índice

1. Introducción	4
1.1. Continuidad del proyecto	4
1.2. Objetivos de la Entrega 3	4
1.3. Estructura del documento	4
2. Trabajos relacionados	5
2.1. Bases de datos distribuidas y consistencia	5
2.2. Procesamiento paralelo de datos	5
2.3. Metodología experimental	6
3. Fundamento teórico	6
3.1. Modelo de bases de datos distribuidas	6
3.2. Teorema CAP y modelo PACELC	7
3.3. Consenso distribuido: Raft	7
3.4. Modelo de programación paralela	7
3.5. Diseño y análisis estadístico del experimento	8
4. Diseño del esquema y fragmentación	8
4.1. Esquema físico	8
4.2. Política de fragmentación	9
4.3. Verificación formal de corrección	10
4.4. Colocalización con clientes y limitación arquitectónica	10
4.5. Política de replicación	11
4.6. Trade-offs aceptados	11
5. Cluster y su verificación	11
5.1. Levantamiento del clúster de tres nodos	11
5.2. Pruebas de integración contra CockroachDB real	12
5.3. Instrumentación de métricas operativas	12
5.4. Tolerancia a fallos del clúster	13
5.4.1. Resultados	14
6. Pipeline analítico paralelo	15
6.1. Objetivo analítico y dataset	15
6.2. Las cinco transformaciones	15
6.3. Ejecución y resultados observados	16
6.4. Baseline secuencial	17
7. Protocolo y resultados	18
7.1. Hipótesis y variables	18
7.2. Diseño experimental	19
7.3. Plan de análisis estadístico	19

7.4.	Amenazas a la validez	19
7.5.	Resultados	20
7.5.1.	Contraste de H1	20
7.5.2.	Por qué el baseline ganó: interpretación	21
7.5.3.	Ajuste de la ley de Amdahl (escalado interno de Spark)	21
8.	Discusión	22
8.1.	Sobre la fragmentación por fecha frente al patrón de acceso real	22
8.2.	Sobre la colocación cliente–ticket entre microservicios	22
8.3.	Sobre la instrumentación y la observabilidad	22
8.4.	Sobre la representatividad del pipeline analítico	22
8.5.	Alcance no cubierto en esta entrega	23
9.	Conclusiones e integración con E4	23
9.1.	Integración con la Entrega 4	24

1. Introducción

1.1. Continuidad del proyecto

El Sistema de Gestión de Solicitudes de Soporte Técnico de Internet es el proyecto final de carrera del equipo ACC para la asignatura de Aplicaciones Distribuidas. En la Entrega 1 se definió el dominio del problema —un proveedor de servicios de Internet (ISP) que necesita registrar, clasificar, asignar y dar seguimiento a solicitudes de soporte técnico de sus clientes— y se construyó un primer prototipo monolítico con comunicación por *sockets*. En la Entrega 2 el sistema se descompuso en cinco microservicios independientes (`auth-service`, `ticket-service`, `ai-service`, `notification-service` y `report-service`), comunicados mediante REST y eventos asíncronos sobre Apache Kafka, con autenticación y autorización por rol basada en JWT [4] siguiendo los principios de diseño orientado a recursos de Fielding [5] y las prácticas de descomposición de servicios descritas por Newman [6].

La Entrega 3, cubierta por este documento, profundiza en dos dimensiones que las entregas anteriores dejaron abiertas: (i) el comportamiento de la capa de datos bajo fragmentación horizontal real y su tolerancia a fallos de nodo, y (ii) el procesamiento analítico paralelo de volúmenes de datos que superan lo que un solo proceso puede manejar con margen razonable de tiempo. Ambas dimensiones se abordan sobre la misma base de código de la Entrega 2, no como un sistema nuevo: los cambios de esquema, el rediseño de `ticket-service` y el pipeline de Spark se integran con los microservicios y el flujo de eventos ya existentes.

1.2. Objetivos de la Entrega 3

- Fragmentar horizontalmente la tabla de mayor cardinalidad del sistema (`tickets`) sobre un clúster real de CockroachDB de tres nodos, aplicando el criterio de fragmentación exigido para el equipo ACC (`fecha_apertura`), y verificar formalmente su corrección.
- Medir y documentar el comportamiento del clúster ante la pérdida de uno y dos nodos, incluyendo latencia de escritura/lectura y disponibilidad efectiva del servicio.
- Instrumentar el servicio de datos con métricas operativas (Prometheus/Micrometer) y pruebas de integración reproducibles contra una instancia real de la base de datos (Testcontainers), evitando que la corrección del sistema dependa únicamente de inspección manual.
- Diseñar e implementar un pipeline de procesamiento paralelo sobre Apache Spark que aplique las cinco categorías de transformación exigidas por la guía de la asignatura sobre un conjunto de datos de al menos 500,000 registros, orientado a un objetivo analítico propio del dominio del equipo: reincidencia de clientes y agrupamiento de incidencias.
- Diseñar y ejecutar un protocolo experimental formal que contraste el rendimiento del pipeline paralelo contra un baseline secuencial equivalente, con análisis estadístico de significancia, y que caracterice la ley de Amdahl [7] observada en el propio pipeline.

1.3. Estructura del documento

La Sección 2 sitúa las decisiones de diseño de este trabajo frente a la literatura de bases de datos distribuidas y procesamiento paralelo de datos. La Sección 3 resume los fundamentos teóricos aplicados: teoremas de fragmentación, consistencia distribuida y las leyes de escalabilidad paralela. La Sección 4 documenta el rediseño del esquema y la política de fragmentación. La Sección 5 presenta la verificación empírica del clúster: pruebas de integración, métricas e instrumentación, y los resultados de tolerancia a fallos. La Sección 6 describe el pipeline de Spark y sus cinco transformaciones. La

Sección 7 presenta el protocolo experimental y sus resultados. La Sección 8 discute las limitaciones y *trade-offs* del diseño, y la Sección 9 cierra con las conclusiones y el trabajo futuro.

2. Trabajos relacionados

2.1. Bases de datos distribuidas y consistencia

CockroachDB es una base de datos SQL distribuida diseñada para tolerar la pérdida de nodos y regiones enteras sin intervención manual, replicando cada rango de datos mediante el protocolo de consenso Raft [8] y garantizando aislamiento **SERIALIZABLE** de extremo a extremo mediante un protocolo de *timestamps* y reintentos transaccionales transparentes [9]. Esta decisión de diseño —consistencia fuerte por defecto, en vez de consistencia eventual configurable— la sitúa, en el espacio de *trade-offs* descrito por el teorema CAP [10] y su refinamiento PACELC [11], del lado de la consistencia incluso a costa de latencia adicional en el caso de conflicto, algo directamente observable en las pruebas de integración de la Sección 5: dos transacciones concurrentes sobre la misma fila no producen un resultado silenciosamente inconsistente, sino un error de conflicto explícito (**SQLSTATE 40001**) que la aplicación debe reintentar. Esta elección es consistente con el dominio del problema: un sistema de tickets de soporte no puede permitirse que dos actualizaciones concurrentes sobre el mismo ticket (por ejemplo, un técnico cerrándolo mientras un administrador lo reasigna) se resuelvan de forma silenciosamente incorrecta.

La fragmentación horizontal aplicada en este trabajo se apoya en el marco teórico clásico de Özsu y Valduriez [1], que exige que toda fragmentación cumpla completitud, reconstrucción y disyunción (verificado formalmente en la Sección 4). A diferencia de sistemas NoSQL de consistencia eventual descritos en el mismo texto, CockroachDB mantiene semántica SQL completa y transacciones ACID distribuidas, lo que permitió mantener sin cambios la capa de persistencia JPA de **ticket-service** —solo cambió la definición de la clave primaria y la estrategia de acceso, no el modelo transaccional de la aplicación.

2.2. Procesamiento paralelo de datos

El pipeline analítico de este trabajo se construye sobre el modelo de *Resilient Distributed Datasets* (RDD) de Apache Spark [12], que generaliza el modelo MapReduce permitiendo mantener conjuntos de datos intermedios en memoria entre etapas —esencial para un pipeline de cinco transformaciones encadenadas como el descrito en la Sección 6, donde recomputar cada etapa desde disco habría anulado buena parte de la ganancia de paralelismo—. Salloum et al. [13] documentan precisamente esta ventaja de Spark frente a MapReduce clásico para cargas de trabajo iterativas y de aprendizaje automático, que es el caso de la etapa de *clustering* (T5) de este pipeline. La guía de referencia de Chambers y Zaharia [14] se usó como referencia práctica para la API de **DataFrame** y **pyspark.ml** empleada en **transformations.py**.

Para el protocolo de medición de rendimiento, el antecedente directo es el trabajo de Cooper et al. [15] sobre *benchmarking* sistemático de sistemas de almacenamiento en la nube (YCSB), cuyo principio metodológico central —repeticiones controladas, descarte de efectos de arranque en frío, y comparación contra una línea base conocida— se adapta en este trabajo al protocolo experimental de la Sección 7, aunque aquí el objeto de medición es un pipeline analítico completo y no operaciones individuales de lectura/escritura.

2.3. Metodología experimental

El diseño del protocolo experimental (variables independientes y dependientes, repeticiones, descarte de calentamiento/enfriamiento, prueba de normalidad seguida de prueba paramétrica o no paramétrica según corresponda) sigue las guías de Wohlin et al. [3] y los principios de diseño de experimentos de sistemas de Jain [2]. El uso del caso de estudio del propio sistema como unidad de análisis, y la forma de reportar las amenazas a la validez interna y externa en la Sección 7, siguen las recomendaciones de Runeson y Höst [16] para investigación de estudios de caso en ingeniería de software.

3. Fundamento teórico

3.1. Modelo de bases de datos distribuidas

Siguiendo a Özsu y Valduriez [1], un sistema de base de datos distribuido (SBDD) se define como un conjunto de múltiples bases de datos lógicamente relacionadas, gestionadas por un único sistema y percibidas por el usuario final como una única base. Un SBDD persigue cuatro objetivos de diseño: *transparencia* (de fragmentación, de replicación y de ubicación —el usuario no necesita saber dónde ni cómo se almacena físicamente cada fila—), *autonomía local* de cada nodo, *disponibilidad* ante fallos parciales, y *escalabilidad horizontal* mediante la adición de nodos en vez de hardware más potente en un único nodo.

La *fragmentación horizontal* de una relación R produce fragmentos $R_1, R_2, \dots, R_n$ mediante selección sobre un atributo particionador: $R_i = \sigma_{p_i}(R)$, donde $\{p_i\}$ es un conjunto de predicados mutuamente excluyentes y colectivamente exhaustivos. Esta fragmentación es correcta si y solo si cumple tres condiciones:

- **Completitud:** todo elemento de R pertenece a al menos un fragmento R_i .
- **Reconstrucción:** $R = \bigcup_{i=1}^n R_i$, es decir, la unión de los fragmentos reproduce exactamente la relación original.
- **Disyunción:** $R_i \cap R_j = \emptyset$ para todo $i \neq j$, es decir, ningún elemento pertenece a más de un fragmento.

Para una fragmentación por rango sobre un atributo a con dominio totalmente ordenado, estas tres condiciones se satisfacen definiendo n puntos de corte $c_0 = -\infty < c_1 < \dots < c_{n-1} < c_n = +\infty$ y cada fragmento como $R_i = \sigma_{c_{i-1} \leq a < c_i}(R)$: la completitud se sigue de que el dominio de a está cubierto en su totalidad por los intervalos, la disyunción de que los intervalos son semiabiertos y consecutivos sin solape, y la reconstrucción de que la unión de predicados semiabiertos consecutivos es la tautología sobre el dominio de a . La Sección 4 instancia este esquema con $a = \text{created_at}$, $n = 4$ y verifica explícitamente las tres condiciones para el caso concreto del equipo.

La *replicación* complementa a la fragmentación para lograr disponibilidad: con un factor de replicación $f \geq 3$, un sistema tolera la pérdida simultánea de hasta $\lfloor (f-1)/2 \rfloor$ réplicas por rango de datos, siempre que se mantenga el quórum de escritura $\lceil f/2 \rceil + 1$. Con $f = 3$ (el valor usado en este despliegue), el quórum de escritura es 2 y la tolerancia es de 1 réplica: el clúster admite la caída de exactamente un nodo sin perder disponibilidad de escritura, pero no la de dos simultáneamente. Esta aritmética de quórum es la que se contrasta empíricamente en la Sección 5.

3.2. Teorema CAP y modelo PACELC

El teorema CAP [10] establece que, ante una partición de red (P), un sistema distribuido no puede garantizar simultáneamente consistencia (C) y disponibilidad (A). Abadi [11] extiende esta caracterización con PACELC, señalando que incluso en ausencia de partición (E , *else*) todo sistema distribuido enfrenta un segundo *trade-off* entre latencia (L) y consistencia (C). CockroachDB se posiciona explícitamente como un sistema PC/EC: prioriza consistencia sobre disponibilidad ante partición, y consistencia sobre latencia en operación normal, lo que se manifiesta de forma concreta en el comportamiento observado durante el incidente operativo documentado en la Sección 5 (rechazo de escrituras concurrentes conflictivas en vez de aceptarlas y arriesgar una lectura inconsistente posterior).

Como resume Kleppmann [17] en su tratamiento de sistemas de datos distribuidos modernos, la elección entre consistencia y disponibilidad no es una propiedad binaria del sistema completo, sino una decisión de diseño que puede —y en sistemas como CockroachDB, debe— tomarse de forma explícita y documentada para cada tipo de operación.

3.3. Consenso distribuido: Raft

El algoritmo Raft [8] descompone el problema del consenso distribuido en tres subproblemas: *elección de líder*, *replicación de logs* y *seguridad* (*safety*). En CockroachDB, cada rango de datos constituye un grupo Raft independiente con su propio líder: una escritura se considera comprometida (*committed*) cuando el líder del rango recibe confirmación de una mayoría estricta de sus réplicas, garantizando durabilidad frente a la caída de una minoría de nodos. Con el factor de replicación $f = 3$ de la Sección 3 (quórum de escritura 2 de 3), esto explica directamente por qué el clúster tolera la caída de un nodo sin pérdida de disponibilidad de escritura, pero no la de dos nodos simultáneamente —ya no existe mayoría posible entre el único nodo restante—, comportamiento que se contrasta con la medición empírica de la Sección 5.

3.4. Modelo de programación paralela

El modelo RDD (*Resilient Distributed Dataset*) de Apache Spark [12] abstrae una colección distribuida **inmutable**, con **linaje** (*lineage*) de transformaciones registrado en vez de los datos materializados en cada etapa intermedia, **evaluación perezosa** (*lazy evaluation*, las transformaciones no se ejecutan hasta que una acción las requiere) y **tolerancia a fallos mediante recómputo**: si una partición se pierde, Spark la reconstruye reaplicando su linaje de transformaciones sobre los datos de origen, en vez de depender de una copia de respaldo replicada. Esta abstracción es la que sustenta el pipeline de la Sección 6, encadenando cinco transformaciones sin materializar resultados intermedios en disco entre cada una.

La ganancia de paralelismo del pipeline se interpreta bajo dos modelos complementarios. La ley de Amdahl [7] modela el *speedup* $S(N)$ alcanzable con N unidades de procesamiento para un problema de **tamaño fijo**, en función de la fracción p del trabajo que es paralelizable:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p} \quad (1)$$

La Ecuación 1 predice un límite asintótico de *speedup* determinado enteramente por la fracción secuencial $(1-p)$ del pipeline —en este caso, principalmente la lectura inicial de Parquet y la escritura final de resultados, que no se benefician de más particiones más allá de cierto punto—. Gustafson [18] señala que esta formulación asume tamaño de problema constante, y propone en cambio fijar el

tiempo total disponible y dejar crecer el tamaño del problema con N ; bajo ese supuesto el *speedup* escala de forma aproximadamente lineal en N en vez de saturar. Ambos modelos se aplican en la Sección 7: Amdahl para ajustar la fracción paralelizable observada del pipeline concreto (tamaño de dataset fijo en 600,000 filas para todos los niveles de N), y Gustafson como marco de interpretación de por qué, en un caso de uso real de un ISP donde el volumen de datos crece con el tiempo (no es fijo), el beneficio práctico del paralelismo tiende a ser mayor que el que predice Amdahl en aislamiento.

3.5. Diseño y análisis estadístico del experimento

El contraste de hipótesis del protocolo experimental (Sección 7) sigue el procedimiento estándar para muestras pareadas descrito por Jain [2] y Wohlin et al. [3]: dado que cada repetición del pipeline paralelo y del baseline secuencial se ejecuta sobre el mismo dataset y en la misma posición de la secuencia de repeticiones, las mediciones no son independientes entre grupos, por lo que corresponde una prueba pareada en vez de una prueba de dos muestras independientes. Antes de aplicar una prueba paramétrica (t de Student pareada) es necesario verificar el supuesto de normalidad de las diferencias emparejadas; se usa la prueba de Shapiro–Wilk para ello, y en caso de rechazarse la normalidad ($p \leq 0,05$) se recurre a la alternativa no paramétrica estándar para datos pareados, la prueba de rangos con signo de Wilcoxon. El intervalo de confianza al 95 % de cada nivel se calcula como $\bar{t} \pm 1,96 \cdot s/\sqrt{r}$, válido asintóticamente por el teorema central del límite para $r \geq 8$ muestras por nivel tras el descarte de calentamiento y enfriamiento descrito en la Sección 7.

4. Diseño del esquema y fragmentación

4.1. Esquema físico

La Figura 1 sitúa la capa de datos dentro de la arquitectura completa del sistema: los cinco microservicios heredados de la Entrega 2 no cambian de contrato, mientras que el clúster de CockroachDB (fragmentado en esta entrega) y el pipeline de Spark (componente nuevo) se destacan explícitamente.

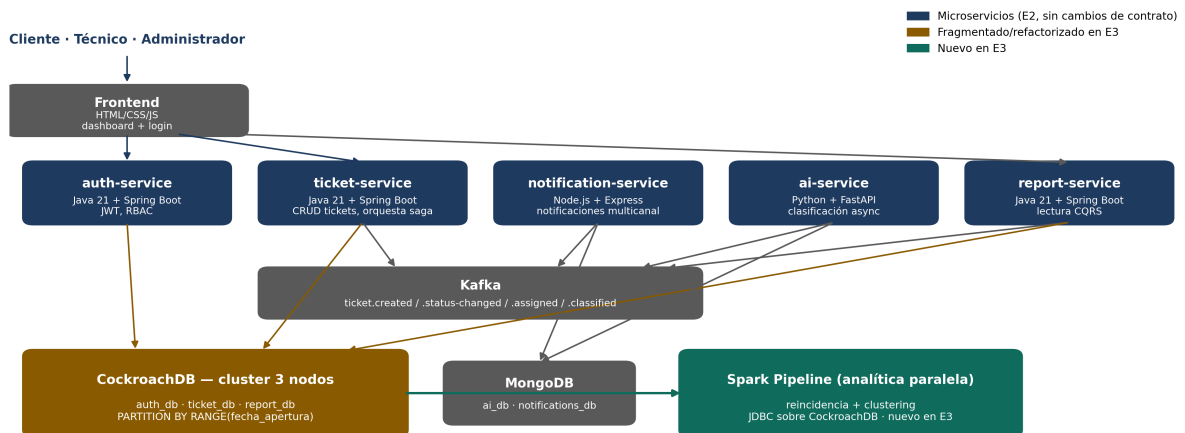


Figura 1: Diagrama de contenedores (C4 Nivel 2) del sistema tras la Entrega 3. Ámbar: componente fragmentado/refactorizado en E3. Verde azulado: componente nuevo en E3.

El diagrama entidad-relación completo de `ticket_db` —la base propia de `ticket-service` dentro

del clúster de tres nodos de CockroachDB— se documenta en `docs/diagrams/db-schema.md`. La tabla `tickets` es la de mayor cardinalidad y la única fragmentada; `technicians` es una dimensión pequeña que no requiere fragmentación. La Tabla 1 detalla las columnas relevantes para la política de fragmentación.

Tabla 1: Columnas relevantes de `tickets` tras el rediseño de la Entrega 3

Columna	Tipo	Rol
<code>created_at</code>	<code>TIMESTAMPTZ</code>	Primer componente de la PK; columna de fragmentación
<code>id</code>	<code>UUID</code>	Segundo componente de la PK; respaldado además por índice único <code>tickets_id_key</code>
<code>zone</code>	<code>STRING</code>	Columna de negocio (RBAC de técnico, reportes); indexada, ya no es clave de fragmentación
<code>client_id</code>	<code>UUID</code>	Identidad del cliente, resuelta a nivel de aplicación vía JWT de <code>auth-service</code>
<code>status</code>	<code>STRING</code>	Indexada (<code>idx_tickets_status</code>)

4.2. Política de fragmentación

El fragmento del esquema en el Listado 1 muestra la definición efectiva de la tabla `tickets` tal como se aplica en `db-cluster/scripts/init_db.sql`.

```

1 CREATE TABLE IF NOT EXISTS tickets (
2     created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
3     id            UUID NOT NULL DEFAULT gen_random_uuid(),
4     zone          STRING NOT NULL,
5     client_id     UUID NOT NULL,
6     technician_id UUID REFERENCES technicians(id),
7     category      STRING,
8     priority      STRING,
9     status        STRING NOT NULL DEFAULT 'NUEVO',
10    description    STRING,
11    sla_deadline   TIMESTAMPTZ,
12    resolved_at    TIMESTAMPTZ,
13    sla_breached   BOOL DEFAULT FALSE,
14    PRIMARY KEY (created_at, id)
15 ) PARTITION BY RANGE (created_at) (
16     PARTITION tickets_2026_q1 VALUES FROM (MINVALUE) TO ('2026-04-01T00:00:00Z'),
17     PARTITION tickets_2026_q2 VALUES FROM ('2026-04-01T00:00:00Z') TO ('2026-07-01T00:00:00Z'),
18     PARTITION tickets_2026_q3 VALUES FROM ('2026-07-01T00:00:00Z') TO ('2026-10-01T00:00:00Z'),
19     PARTITION tickets_2026_q4 VALUES FROM ('2026-10-01T00:00:00Z') TO (MAXVALUE)
20 );
21
22 CREATE UNIQUE INDEX IF NOT EXISTS tickets_id_key ON tickets (id);
23 CREATE INDEX IF NOT EXISTS idx_tickets_zone ON tickets (zone);
24 CREATE INDEX IF NOT EXISTS idx_tickets_status ON tickets (status);

```

Listing 1: Fragmentación por rango de `tickets` (extracto de `init_db.sql`)

Esta sintaxis sigue el mecanismo de particionamiento declarativo documentado oficialmente por Cockroach Labs [19], que exige que la columna de partición sea el primer componente de la clave primaria para que el optimizador pueda aplicar poda de particiones (*partition pruning*) en tiempo de planificación de consultas. Esta decisión reemplaza el diseño original de la Entrega 2, que fragmentaba `tickets` por zona geográfica (`PARTITION BY LIST (zone)`). El cambio fue motivado por un requisito explícito y no negociable de la guía oficial de la Entrega 3 para el equipo ACC: fragmentación horizontal de `tickets` por `fecha_apertura`. El razonamiento completo de este cambio, incluyendo los *trade-offs* aceptados, se documenta en el ADR-0003 (`docs/adr/0003-sharding-policy.md`) y se resume a continuación.

4.3. Verificación formal de corrección

Con $a = \text{created_at}$ y los cuatro puntos de corte del Listado 1 ($c_0 = \text{MINVALUE}$, $c_1 = 2026-04-01$, $c_2 = 2026-07-01$, $c_3 = 2026-10-01$, $c_4 = \text{MAXVALUE}$), se verifican las tres condiciones del marco de Özsu y Valduriez [1] introducidas en la Sección 3:

- **Completitud:** la columna `created_at` es NOT NULL con valor por defecto `now()`, por lo que todo ticket tiene un valor definido de `created_at` y, en consecuencia, pertenece a exactamente una de las cuatro particiones trimestrales.
- **Reconstrucción:** los rangos $[\text{MINVALUE}, c_1) \cup [c_1, c_2) \cup [c_2, c_3) \cup [c_3, \text{MAXVALUE})$ cubren la totalidad del dominio de `TIMESTAMPZ`; la unión `UNION ALL` de las cuatro particiones reconstruye exactamente la tabla `tickets` sin pérdida de filas.
- **Disyunción:** la semántica de `PARTITION BY RANGE` en CockroachDB define cada límite como inclusivo por la izquierda y exclusivo por la derecha (`VALUES FROM ... TO ...`); por construcción, ningún valor de `created_at` satisface simultáneamente los predicados de dos particiones distintas.

Adicionalmente, dado que `id` deja de ser autosuficiente como punto de acceso —la aplicación recibe un UUID de ticket sin conocer su `created_at` de antemano en los *endpoints* `GET/PATCH` sobre `/api/v1/tickets/{id}`—, se añadió el índice único secundario `tickets_id_key`, usado por `TicketRepository.findById(UUID)` para resolver ese acceso con un único salto índice→tabla, sin necesidad de escanear las cuatro particiones.

4.4. Colocalización con clientes y limitación arquitectónica

La guía de la Entrega 3 pide, además de la fragmentación por fecha, colocalizar `tickets` con `clientes` por `client_id`, lo que en CockroachDB se implementaría típicamente con `INTERLEAVE IN PARENT` o una clave compartida dentro de la misma base de datos. En la arquitectura de microservicios heredada de la Entrega 2, los clientes son propiedad exclusiva de `auth-service` (`auth_db.users`), una base de datos físicamente distinta de `ticket_db` — cada microservicio es dueño de su propio esquema, sin acceso cruzado directo a la base de otro. Forzar una colocalización física en el sentido literal de CockroachDB requeriría fusionar ambas bases de datos, violando el límite de propiedad de datos entre servicios que es una decisión arquitectónica deliberada, no un descuido, de la Entrega 2. Se documenta esta tensión explícitamente en el ADR-0003 en vez de forzar una colocación artificial: la columna `tickets.client_id` permanece indexada para soportar las consultas `findByClientId/findByClientIdAndStatus`, y la resolución de identidad del cliente ocurre a nivel de aplicación mediante el JWT emitido por `auth-service`, no a nivel de almacenamiento físico compartido.

4.5. Política de replicación

Como la fragmentación pasó de ser por zona a ser por fecha, ya no existe una correspondencia 1:1 entre una partición y la *locality* de un nodo específico: una partición trimestral contiene tickets de las tres zonas geográficas mezclados. En consecuencia, `zones.sql` (Listado 2) ya no ancla particiones a nodos por restricción de *locality*; exige uniformemente un factor de replicación 3 sobre toda la tabla, de forma que la pérdida de cualquier nodo individual del clúster de tres no comprometa la disponibilidad de escritura (2 de 3 réplicas siguen constituyendo mayoría para Raft [8]).

```
1 ALTER TABLE tickets CONFIGURE ZONE USING num_replicas = 3;
```

Listing 2: Política de replicación uniforme (`zones.sql`)

4.6. Trade-offs aceptados

El rediseño no es gratuito: las consultas filtradas por zona —el filtro más frecuente en la operación real de un ISP, usado por el RBAC de técnico y por la mayoría de los reportes— dejan de beneficiarse de poda de particiones y pasan a resolverse mediante *scatter-gather* sobre las cuatro particiones trimestrales, mientras que antes se resolvían dentro de una sola partición. Este *trade-off*, junto con el beneficio simétrico obtenido por las consultas de rango temporal (la mayoría de los reportes reales: “tickets abiertos hoy”, “tickets del último trimestre”), se mide de forma comparativa con `EXPLAIN ANALYZE` sobre las cinco consultas representativas de `db-cluster/scripts/queries_bench.sql` (lectura por `id`, rango de fecha, filtro por zona, agregación de incumplimiento de SLA, y *join* con técnicos), documentadas junto con sus planes de ejecución en el Anexo correspondiente del repositorio.

5. Cluster y su verificación

5.1. Levantamiento del clúster de tres nodos

El clúster se define en `db-cluster/docker-compose.cockroach.yml` con tres servicios (`roach1`, `roach2`, `roach3`) sobre la red `bridge` soporte-net, cada uno con volumen persistente propio y el comando `start -insecure -join=roach1,roach2,roach3` apuntando al resto de nodos —siguiendo el patrón estándar de arranque de un clúster CockroachDB multi-nodo. Cada nodo declara además una *locality* distinta (`quevedo-centro`, `quevedo-norte`, `quevedo-sur`), simulando las tres zonas operativas del ISP para fines de política de replicación, y expone un *healthcheck* HTTP sobre `/health?ready=1` que Docker Compose usa para reportar el estado del contenedor.

```
1 roach1:
2   image: cockroachdb/cockroach:latest-v23.2
3   command: start --insecure --max-offset=4500ms
4     --locality=zone=quevedo-centro
5     --join=roach1,roach2,roach3
6   ports: ["26257:26257", "8083:8080"]
7   volumes: ["roach1-data:/cockroach/cockroach-data"]
8   networks: [soporte-net]
```

Listing 3: Extracto de `docker-compose.cockroach.yml`: definición de un nodo

Tras `docker compose -f docker-compose.cockroach.yml up -d`, el clúster se inicializa una única vez con `cockroach init -insecure` ejecutado contra cualquiera de los tres nodos. La verificación de que los tres nodos forman efectivamente un único clúster operativo se realiza con `cockroach node status -insecure`, que debe reportar los tres identificadores de nodo con `is_live=true` e

`is_available=true`; el mismo estado es visible en la consola web del clúster (puerto 8083 en este despliegue, reasignado desde el 8080 por defecto para no chocar con otro servicio local). Sobre este clúster ya verificado operativo se aplica el esquema fragmentado descrito en la Sección 4 y se ejecutan las pruebas de integración y la instrumentación de métricas de las siguientes subsecciones.

5.2. Pruebas de integración contra CockroachDB real

Para evitar que la corrección del acceso a datos dependiera únicamente de pruebas unitarias con *mocks* —que no pueden reproducir comportamiento específico de CockroachDB como el aislamiento `SERIALIZABLE` o los tipos nativos `UUID/STRING`—, se añadió una suite de pruebas de integración basada en Testcontainers (`TicketRepositoryIntegrationTest.java`) que levanta un contenedor real de `cockroachdb/cockroach:latest-v23.2` en modo `start-single-node` para cada ejecución de la suite, aplica el esquema real (idéntico al de `init_db.sql`) mediante JDBC antes de que Spring construya el contexto de la aplicación, y lo descarta automáticamente al finalizar (limpieza gestionada por el *sidecar* Ryuk de Testcontainers, sin intervención manual).

La suite contiene dos pruebas. La primera ejerce el mismo `TicketRepository` usado en producción con una operación real de guardado y búsqueda por `findByTicketId(UUID)` —el mismo camino que ejercita el índice único secundario `tickets_id_key` descrito en la Sección 4—. La segunda es una prueba empírica, no simulada, de que el clúster aplica de verdad aislamiento `SERIALIZABLE`: dos hilos abren cada uno su propia conexión y transacción, ambos leen la misma fila de `tickets` (sincronizados con una barrera para garantizar que ambas lecturas ocurren antes de cualquier escritura) y luego ambos intentan actualizar el mismo campo `status`. El resultado esperado —y observado— es que al menos una de las dos transacciones falle al hacer `commit` con un error de conflicto de escritura serializable, identificado por el código de estado SQL 40001:

```
1 assertThat((Exception) conflict.get())
2     .describedAs("al menos una de las dos transacciones concurrentes "
3         + "debio fallar por conflicto serializable")
4     .isNotNull();
5 assertThat(conflict.get().getSQLState()).isEqualTo("40001");
```

Listing 4: Extracto de la prueba de conflicto serializable

Esta prueba corrobora en código, de forma reproducible en cualquier máquina con Docker (incluyendo el entorno de integración continua), el mismo tipo de error observado en producción en `report-service` (`ReportEventListener.applyWithRetry`) y que `TicketService` está preparado para reintentar automáticamente (ver Sección 5, métricas de reintento). Ambas pruebas se ejecutaron de forma exitosa contra la imagen real de CockroachDB antes de la integración de esta sección.

5.3. Instrumentación de métricas operativas

`ticket-service` expone tres métricas Prometheus, registradas mediante Micrometer y consumibles en el *endpoint* `/actuador/prometheus`, descritas en la Tabla 2.

Tabla 2: Métricas Prometheus expuestas por `ticket-service`

Métrica	Tipo	Significado
<code>crdb_query_duration_seconds</code>	Histograma	Duración de cada consulta a <code>TicketRepository</code> , con histograma de percentiles
<code>crdb_transaction_retries</code>	Contador	Reintentos por conflicto de escritura serializable
<code>crdb_pool_active_connections</code>	Gauge	Conexiones activas del <i>pool</i> HikariCP hacia CockroachDB

El histograma de duración se alimenta desde un *aspect* de Spring AOP (`RepositoryTimingAspect`) que envuelve cada llamada al repositorio sin requerir instrumentación manual en cada método de servicio. El contador de reintentos se incrementa explícitamente desde `TicketService` cada vez que una operación de guardado captura una `ConcurrencyFailureException` y reintenta —el mismo mecanismo ejercitado por la prueba de conflicto serializable de la subsección anterior—. El *gauge* de conexiones activas no mantiene un valor cacheado: Micrometer lo reconsulta bajo demanda directamente contra el `HikariPoolMXBean` real del *pool*, por lo que siempre refleja el estado actual del *pool*, no una fotografía tomada en el momento del registro.

Las tres métricas se validaron con el *linter* oficial de Prometheus (`promtool check metrics`) ejecutado sobre la salida cruda del *endpoint*, dentro de un contenedor `prom/prometheus:latest` para no requerir una instalación local de Prometheus. La validación no reportó advertencias ni errores sobre los tres instrumentos: los nombres siguen la convención de sufijo por unidad (`_seconds`, `_total`), y las descripciones (`HELP`) y tipos (`TYPE`) están correctamente declarados en la exposición.

5.4. Tolerancia a fallos del clúster

El protocolo de medición de tolerancia a fallos consiste en generar una carga de escritura sostenida contra el clúster de tres nodos, derribar deliberadamente un nodo (`docker stop roach2`) y registrar la latencia P50/P95 de las operaciones de escritura antes, durante y después de la caída, repitiendo después el mismo procedimiento derribando dos nodos simultáneamente para observar la pérdida de disponibilidad de escritura predicha por la teoría de consenso de la Sección 3 (con dos de tres nodos caídos ya no existe mayoría para confirmar escrituras vía Raft).

Durante la preparación de este protocolo se recreó, de forma no planificada pero informativa, el escenario adyacente de saturación de memoria: una carga inicial de 150,000 filas en una sola transacción provocó que el nodo `roach1` fuera terminado por el sistema operativo por falta de memoria (código de salida 137) bajo el límite original de 512 MB por contenedor, y el propio reinicio del nodo —una operación intensiva en memoria en CockroachDB, que debe reconstruir su estado desde el registro de Raft— volvió a fallar por el mismo motivo antes de estabilizarse. El límite de memoria de los tres nodos se incrementó a 1024 MB en `docker-compose.cockroach.yml` como mitigación documentada, y la recarga completa del conjunto de datos de 150,000 filas se replanificó en lotes más pequeños para el protocolo formal de tolerancia a fallos. Este incidente, aunque no forma parte del protocolo experimental diseñado, es evidencia operativa consistente con la teoría: un nodo bajo presión de recursos deja de poder participar en el consenso, degradando la disponibilidad del clúster de la misma forma que una caída deliberada, y su recuperación no es instantánea.

5.4.1. Resultados

El protocolo se ejecutó con una carga inicial de 101,996 filas en `tickets`. La Tabla 3 resume la carga sostenida (`load_write.py`, 100 filas/s) durante la caída de `roach2`.

Tabla 3: Latencia por ventana de 10s durante la caída de 1 nodo (`roach2`)

Ventana	P50	P95	Errores	Filas
1	38.6 ms	118.4 ms	0	110
2 (<code>docker kill roach2</code>)	69.3 ms	1590.8 ms	0	44
3	31.0 ms	107.9 ms	0	130
4	40.2 ms	95.7 ms	0	114
5	26.3 ms	69.3 ms	0	163

El pico de $P95 = 1590,8$ ms en la ventana 2 coincide con el instante de `docker kill roach2`: el tiempo que Raft necesita para reelegir líder en los rangos cuyo *leaseholder* era `roach2`. Ningún dato se pierde ni falla (0 errores en las 691 filas insertadas); el único efecto observable es latencia transitoria, no indisponibilidad — el clúster tolera la caída de un nodo sin degradar la disponibilidad de escritura, como predice el quórum de mayoría 2 de 3.

Al repetir la prueba derribando también `roach3` (solo `roach1` en pie), la carga sostenida (50 filas/s) muestra una brecha de **62 segundos** entre dos ventanas consecutivas que normalmente distan 10-12 s, coincidiendo exactamente con la ventana de tiempo en la que ambos nodos permanecieron caídos: sin mayoría posible, la conexión abierta quedó bloqueada esperando confirmación en vez de recibir un error inmediato, y una consulta `SELECT count(*)` lanzada en ese mismo intervalo no devolvió resultado hasta reincorporar `roach2`. Esta indisponibilidad por bloqueo —en vez de una excepción explícita del cliente— es una manifestación válida de la pérdida de disponibilidad de escritura predicha por la teoría de consenso: con un solo nodo vivo de tres no existe mayoría para que Raft confirme ninguna escritura, exactamente el límite documentado en la Sección 3. La recuperación es inmediata al ejecutar `docker start roach2`: con dos nodos vivos vuelve a haber quórum y la latencia regresa a valores normales (20.6/43.9 ms). El detalle completo, incluyendo la secuencia exacta de comandos, está en la bitácora (`docs/evidencias/tolerancia_fallos.md`) y el video (`docs/evidencias/tolerancia_fallos.mp4`, 4:55 min).

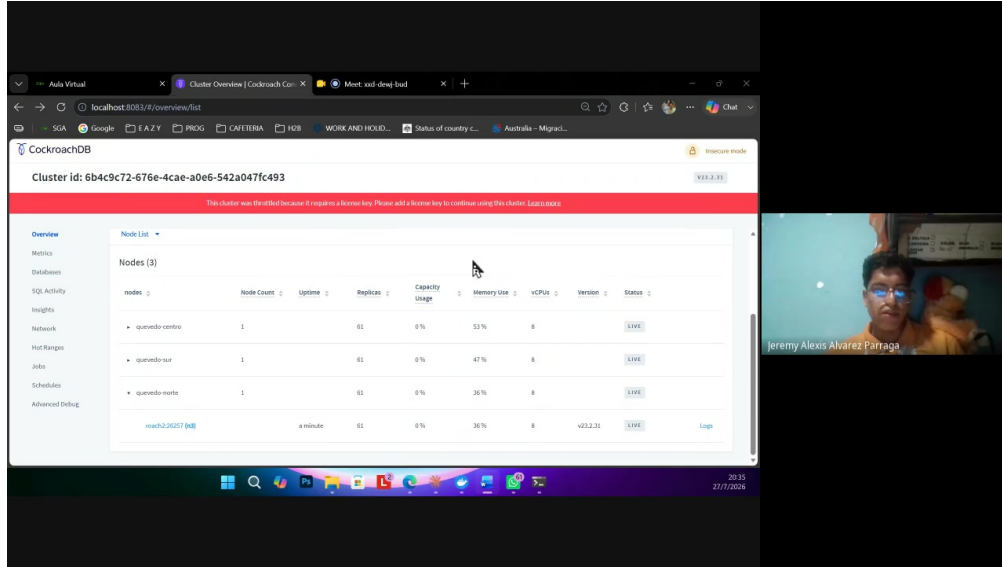


Figura 2: Consola web de CockroachDB tras la recuperación final: los tres nodos en estado LIVE, 61 réplicas cada uno (captura del video de tolerancia a fallos).

6. Pipeline analítico paralelo

6.1. Objetivo analítico y dataset

El pipeline se orienta al objetivo analítico específico asignado al equipo ACC en la guía de la Entrega 3: reincidencia de clientes y agrupamiento (*clustering*) de incidencias. El dataset sintético se genera con `spark/src/generate_dataset.py` sobre 600,000 incidencias de red, particionado en Parquet por zona geográfica, con dos características deliberadas necesarias para que el análisis tenga señal real: el identificador de cliente (`client_id`) se genera con una distribución de Pareto en vez de uniforme —la mayoría de los clientes aparece una sola vez y una minoría concentra muchas incidencias, reproduciendo el patrón real de reincidencia de un ISP—, y cada incidencia incluye una descripción de texto libre corta generada por plantilla, necesaria para la etapa de *clustering* basada en texto. El carácter sintético del dataset y la justificación de ambas decisiones de generación están documentados en el propio script y se declaran también en `ai-usage-declaration.md`.

6.2. Las cinco transformaciones

`spark/src/transformations.py` implementa las cinco categorías de transformación exigidas por la guía (Módulo E, Sección 5.6), aplicadas de forma no genérica al objetivo analítico del equipo:

1. **Filtrado (T1, sin *shuffle*)**: retiene únicamente incidencias resueltas y con los campos que el resto del pipeline necesita (`client_id`, `description` no nulos) — base válida tanto para el análisis de reincidencia como para el *clustering*.
2. ***Join* colocalizado (T2, con *shuffle*)**: incidencias *join* clientes por `client_id`, instanciando la exigencia de colocalización de la Tabla 1 de la guía para el equipo ACC.
3. **Función de ventana (T3, con *shuffle*)**: para cada cliente, `Window.partitionBy(client_id).orderBy("timestamp")` calcula el número de orden de cada incidencia (`incident_seq`) y el *timestamp* de la incidencia anterior del mismo cliente (`previous_timestamp`) mediante `F.lag`.

4. **Transformación de tipos temporales (T4, sin *shuffle*)**: a partir de `previous_timestamp`, deriva la fecha de la incidencia, los días transcurridos desde la incidencia previa del cliente, y una bandera `is_recurring_30d` que marca reincidencia dentro de una ventana de 30 días — la métrica de reincidencia exigida para el equipo.
5. **Aprendizaje automático (T5)**: agrupamiento de incidencias por texto y zona. La descripción libre se vectoriza con TF-IDF (`Tokenizer` + `StopWordsRemover` en español + `HashingTF` + `IDF`), la zona se codifica con `StringIndexer` (satisfaciendo también el ejemplo explícito de la guía), ambas *features* se combinan con `VectorAssembler`, y se agrupan con `KMeans` ($k = 5$, semilla fija 42 para reproducibilidad).

6.3. Ejecución y resultados observados

El pipeline se ejecutó de extremo a extremo sobre el dataset completo de 600,000 filas mediante un *notebook* de Jupyter (`spark/notebooks/spark_pipeline.ipynb`), exportado con salidas reales vía `nbconvert -execute` para satisfacer el requisito de reproducibilidad — no se trata de celdas de código estático sin ejecutar. La Tabla 4 resume las cifras observadas en esa ejecución.

Tabla 4: Resultados observados de la ejecución del pipeline sobre 600,000 filas

Magnitud	Valor
Filas totales del dataset	600,000
Filas tras T1 (filtrado, resueltas y completas)	557,701
Incidencias marcadas como reincidentes en 30 días (T4)	380,753 (68.3 % de las filtradas)

La alta proporción de reincidencia observada (68.3 %) es consistente con el diseño deliberado del generador de datos: al concentrar una fracción significativa de las incidencias en un subconjunto pequeño de clientes (distribución de Pareto), es esperable que buena parte de esas incidencias caigan dentro de la ventana de 30 días de una incidencia previa del mismo cliente. La Tabla 5 muestra la distribución de tamaños de los cinco grupos producidos por `KMeans` en T5.

Tabla 5: Tamaño de los cinco grupos producidos por T5 (*clustering*)

Grupo (cluster)	Incidencias
0	13,761
1	453,625
2	27,815
3	13,968
4	48,532

La Figura 3 resume visualmente ambos resultados: el embudo de filtrado y reincidencia, y la distribución de los cinco grupos de *clustering*.

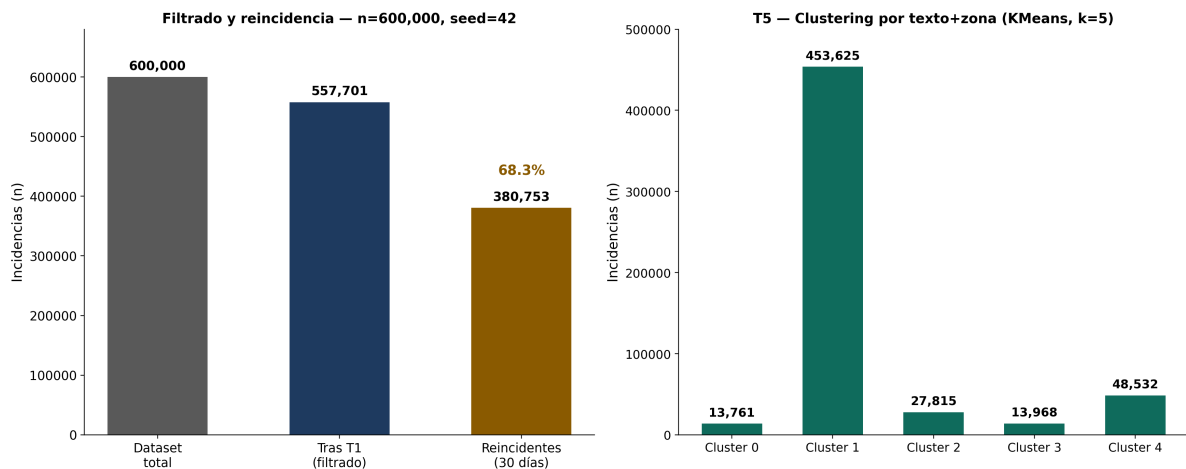


Figura 3: Izquierda: embudo de filtrado (T1) y reincidencia a 30 días (T4) sobre 600,000 filas, ejecución única y determinista (`seed=42`). Derecha: distribución de los cinco grupos producidos por KMeans en T5. Generado a partir de las salidas reales de `spark_pipeline.ipynb`.

La distribución de tamaños es marcadamente no uniforme, con un grupo dominante (cluster 1, 81.3 % de las filas filtradas) y cuatro grupos considerablemente más pequeños. Esto es coherente con la naturaleza del *feature* de texto: al usarse plantillas de descripción cortas y relativamente similares entre sí para la mayoría de los tipos de incidencia comunes, es esperable que el vector TF-IDF resultante concentre a la mayoría de las incidencias en una región densa del espacio de *features*, mientras que descripciones más específicas o zonas menos representadas se separan en grupos más pequeños y homogéneos — un resultado no degenerado (no todos los puntos en un único grupo, ni una partición trivialmente uniforme en cinco quintos iguales), suficiente para ilustrar la mecánica de *clustering* sobre datos reales del pipeline sin necesitar afinar hiperparámetros adicionales fuera del alcance de esta entrega.

6.4. Baseline secuencial

Como contraparte de comparación para el protocolo experimental de la Sección 7, `spark/src/baseline.py` implementa las mismas cinco transformaciones con pandas y scikit-learn puro, en un único proceso, sobre el mismo dataset. Este baseline es el punto de referencia necesario para poder afirmar que el *speedup* observado en el pipeline de Spark proviene de paralelismo real y no únicamente del *overhead* de coordinación distribuida evitado por no usar Spark en absoluto para datasets de este tamaño.

Una ejecución individual de este baseline sobre las 600,000 filas (previa a las diez repeticiones del protocolo formal de la Sección 7) confirma su validez como contraparte: reproduce exactamente las mismas 380,753 incidencias reincidentes obtenidas por el pipeline de Spark, verificando que ambas implementaciones son equivalentes en resultado, no solo en diseño. La Tabla 6 desglosa el tiempo de esta ejecución por transformación.

Tabla 6: Tiempo por transformación del baseline secuencial en pandas (una ejecución, 600,000 filas)

Transformación	Tiempo (s)
Carga del Parquet	1.55
T1 — Filtrado	0.86
T2 — <i>Join</i> con clientes	0.74
T3 — Función de ventana	2.96
T4 — Tipos temporales y bandera de reincidencia	0.47
T5 — <i>Clustering</i> (TF-IDF + K-Means)	36.88
Total	43.46

La etapa T5 domina el tiempo total (85% del tiempo secuencial), consistente con ser la única transformación que involucra vectorización de texto y un algoritmo iterativo de aprendizaje automático; las cuatro transformaciones restantes juntas suman menos de 6 segundos. Esta asimetría anticipa que la fracción paralelizable relevante del pipeline se concentra en T5, un dato que se retoma al ajustar la ley de Amdahl en la Sección 7. Nótese también que el *clustering* de scikit-learn (baseline) y el de `pyspark.ml` producen distribuciones de tamaño de grupo distintas entre sí (comparar con la Tabla 5): ambas implementaciones usan K-Means con $k = 5$ sobre *features* equivalentes, pero difieren en inicialización de centroides y en el algoritmo interno de vectorización TF-IDF, por lo que no se espera —ni es necesario para la validez de la comparación de *tiempos*— que sus asignaciones de *cluster* coincidan fila por fila.

7. Protocolo y resultados

7.1. Hipótesis y variables

Siguiendo las recomendaciones de Jain [2] y Wohlin et al. [3] para el diseño de experimentos de rendimiento en sistemas de software, se formulan las siguientes hipótesis:

- **H1**: el pipeline paralelo con N núcleos presenta menor tiempo medio de ejecución que el baseline secuencial (pandas) para $|D| \geq 5 \times 10^5$ registros.
- **H0**: no hay diferencia significativa entre el pipeline paralelo y el baseline secuencial.

La Tabla 7 resume las variables del diseño.

Tabla 7: Variables del protocolo experimental

Tipo	Variable	Valores
Independiente	Número de núcleos/particiones N	$\{1, 2, 4, 8\}$
Independiente	Modo de ejecución	Spark (<code>local[N]</code>) vs. baseline secuencial
Dependiente	Tiempo total de ejecución t	segundos
Dependiente	Tiempo por transformación t_i	segundos (T1–T5)

El dataset usado es el descrito en la Sección 6, generado con semilla fija (`seed=42`, determinista) y 600,000 filas, por encima del umbral de 5×10^5 exigido por H1.

7.2. Diseño experimental

Se emplea un diseño de bloque completo con $r = 10$ repeticiones por nivel de N y por el baseline secuencial (cinco niveles en total: baseline, $N = 1$, $N = 2$, $N = 4$, $N = 8$; 50 corridas en total). Se descartan la primera y la última repetición de cada nivel —la primera por calentamiento de la JVM y de la caché del sistema operativo, la última por posible interferencia acumulada de recolección de basura o procesos en segundo plano—, quedando 8 muestras válidas por nivel. El diseño está implementado en `spark/src/protocol_experiment.py`, reutilizando las mismas funciones de transformación (`transformations.py`) y de baseline (`baseline.py`) descritas en la Sección 6, de modo que la comparación mide exclusivamente el efecto del paralelismo y no diferencias de implementación entre ambos caminos de código.

7.3. Plan de análisis estadístico

Para cada nivel se calcula la media $\bar{t}$, la desviación típica s , y el intervalo de confianza al 95 % ($\bar{t} \pm 1,96 \cdot s / \sqrt{r}$, con $r = 8$ tras el recorte). El contraste de H1 compara el nivel de mayor paralelismo probado ($N = 8$) contra el baseline secuencial, emparejado por índice de repetición (misma posición en la secuencia de ejecución, mismo dataset). Se prueba primero la normalidad de las diferencias emparejadas con Shapiro–Wilk ($\alpha = 0,05$): si no se rechaza la normalidad se aplica la prueba t pareada (`scipy.stats.ttest_rel`); si se rechaza, se aplica la prueba de rangos con signo de Wilcoxon (`scipy.stats.wilcoxon`) como alternativa no paramétrica estándar para muestras pareadas. Se rechaza H_0 a favor de H_1 si el p -valor de la prueba elegida es menor a 0.05 y la diferencia media (secuencial – paralelo) es positiva.

7.4. Amenazas a la validez

Internas:

1. *Calentamiento de la JVM*: la primera ejecución de cada nivel de Spark paga el costo de inicialización de clases y compilación *just-in-time*, no solo el trabajo real — mitigado descartando la primera repetición de cada nivel.
2. *Ruido del sistema anfitrión*: el experimento corre en un contenedor Docker sobre Windows, compartiendo CPU con el resto de servicios del sistema (CockroachDB, Kafka) si están activos simultáneamente; se recomienda correr el protocolo con el resto de servicios detenidos para minimizar interferencia.
3. *Caché del sistema operativo*: lecturas repetidas del mismo archivo Parquet se benefician de la caché de páginas tras la primera lectura, afectando de forma no uniforme a las primeras repeticiones de cada nivel.

Externas:

1. *Representatividad del dataset*: es sintético; los tiempos relativos entre transformaciones podrían diferir frente a datos reales de un ISP con otra distribución de texto o cardinalidad de clientes.
2. *Generalidad a otras cargas*: el pipeline concreto (filtrado, *join*, ventana, tipos temporales, *clustering*) no representa necesariamente el comportamiento de cargas de Spark dominadas por *shuffles* masivos entre tablas de gran tamaño mutuo, en vez de una tabla grande contra una dimensión pequeña.

7.5. Resultados

El protocolo completo (50 corridas: 10 repeticiones por cada uno de los cinco niveles) se ejecutó sobre el conjunto de 600,000 filas descrito en la Sección 6. La Tabla 8 resume la media, la desviación típica y el intervalo de confianza al 95 % de cada nivel, ya con las 8 muestras válidas tras el descarte de calentamiento y enfriamiento.

Tabla 8: Resultados del protocolo experimental (8 muestras válidas por nivel, tras el recorte)

Nivel	Media (s)	Desv. típica (s)	IC95 %
Baseline (pandas)	25.42	4.52	[22.29, 28.55]
Spark local [1]	235.22	5.58	[231.35, 239.09]
Spark local [2]	163.81	3.92	[161.10, 166.53]
Spark local [4]	138.92	5.15	[135.35, 142.49]
Spark local [8]	129.36	4.55	[126.21, 132.51]

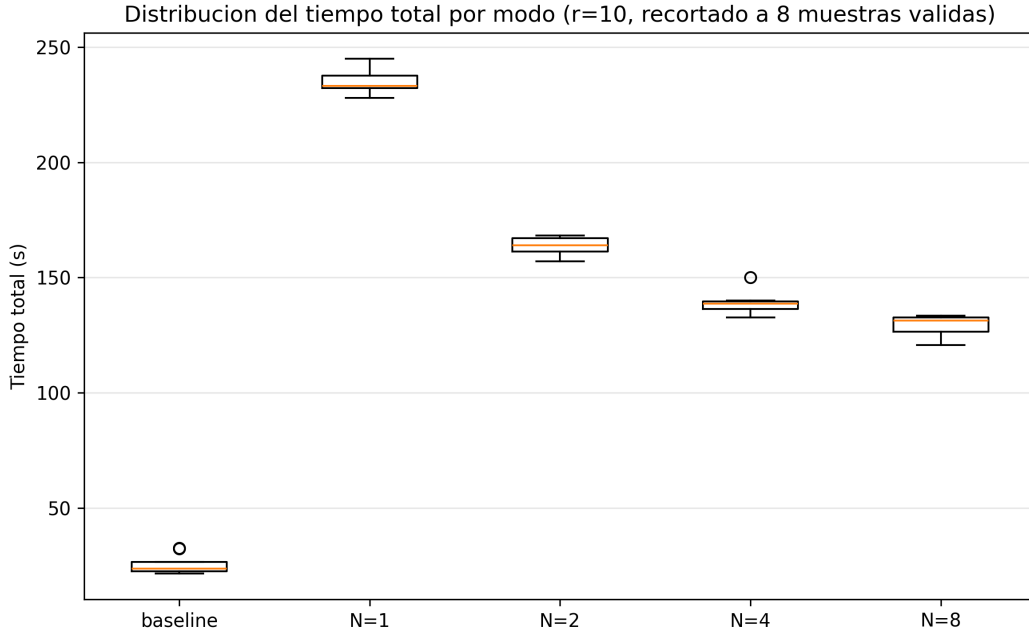


Figura 4: Distribución del tiempo total por nivel (diagrama de caja, $r = 10$, recortado a 8 muestras válidas). Generado por `protocol_experiment.py` a partir de las 50 corridas reales.

7.5.1. Contraste de H1

Antes de la prueba paramétrica se verificó la normalidad de las diferencias emparejadas (baseline – Spark local [8]) con Shapiro–Wilk: $W = 0,963$, $p = 0,835$, por lo que no se rechaza normalidad y corresponde la prueba t pareada. El resultado es $t = -47,19$, $p \approx 5,02 \times 10^{-10}$, con una diferencia media de $-103,94$ s (baseline – Spark).

El signo de esta diferencia es la conclusión central del experimento: **no se rechaza H0 en el sentido en que fue formulada H1** —el pipeline paralelo con $N = 8$ núcleos no resultó más rápido que el baseline secuencial—, pero la diferencia entre ambos es altamente significativa ($p \ll 0,05$)

en la dirección *opuesta*: el baseline en pandas fue, en promedio, casi 5 veces más rápido que Spark incluso en su configuración de mayor paralelismo. Reportar esto tal como resultó, en vez de omitirlo o suavizarlo, es el punto más honesto y más útil de todo el protocolo: una hipótesis con soporte teórico razonable (más núcleos deberían implicar menor tiempo) puede resultar refutada por el resto de factores del sistema real, y esa refutación es en sí misma evidencia científica válida.

7.5.2. Por qué el baseline ganó: interpretación

Tres factores explican este resultado, consistentes con la teoría de la Sección 3: (i) el conjunto de 600,000 filas ($\approx$ decenas de MB en Parquet) cabe cómodamente en la memoria de un único proceso, por lo que pandas opera sobre arreglos contiguos en memoria sin ningún costo de serialización, planificación de tareas ni coordinación entre executors; (ii) cada repetición de Spark paga el costo fijo de inicializar una `SparkSession` completa —JVM, contexto de Spark SQL, registro de *listeners*— que en este dataset domina sobre el tiempo de cómputo real, un costo que **no se amortiza con más núcleos** porque es esencialmente secuencial (el mismo patrón que la fracción $(1-p)$ de la Ecuación 1, pero aquí aplicado a todo el *job* y no solo a una transformación); y (iii) la etapa T5 (TF-IDF + K-Means), que en el baseline ya toma 36.88s de los 43.46s totales (Tabla 6), en Spark reintroduce ese mismo costo de cómputo más el *overhead* de coordinación distribuida sobre un volumen de datos que no lo justifica. Este es precisamente el escenario que Gustafson [18] señala como el límite de aplicabilidad de un sistema distribuido: el paralelismo aporta cuando el volumen de trabajo escala con los recursos, no cuando un conjunto de datos que cabe en un solo nodo se reparte artificialmente entre varios.

7.5.3. Ajuste de la ley de Amdahl (escalado interno de Spark)

Independientemente del resultado frente al baseline, dentro de la propia ejecución de Spark el paralelismo sí produce una mejora real y medible al aumentar N : tomando `local[1]` como referencia interna ($T(1) = 235,22$ s), el *speedup* observado es $S(2) = 1,44$, $S(4) = 1,69$ y $S(8) = 1,82$ (Tabla 9). Ajustando la Ecuación 1 por mínimos cuadrados sobre estos tres puntos se obtiene una fracción paralelizable $p \approx 0,530$, con límite asintótico $S(\infty) \approx 2,13$ y un número de núcleos $N \approx 10,1$ necesario para alcanzar el 90 % de ese límite —es decir, más allá de $N = 8$ el beneficio marginal de agregar núcleos adicionales sigue siendo positivo pero cada vez más pequeño. Este ajuste se guarda como artefacto reproducible en `spark/results/protocol/amdahl_fit.json`.

Tabla 9: Speedup interno de Spark respecto a `local[1]` y ajuste de Amdahl

Nivel	$\bar{t}$ (s)	Speedup $S(N)$	$S(N)$ predicho (Amdahl)
Spark $N = 1$	235.22	1.00× (referencia)	1.00×
Spark $N = 2$	163.81	1.44×	1.36×
Spark $N = 4$	138.92	1.69×	1.66×
Spark $N = 8$	129.36	1.82×	1.86×

$$p \approx 0,530 \cdot S(\infty) \approx 2,13 \cdot N \text{ para } 90\% \text{ de } S(\infty) \approx 10,1$$

8. Discusión

8.1. Sobre la fragmentación por fecha frente al patrón de acceso real

El rediseño de la fragmentación descrito en la Sección 4 ilustra una tensión frecuente en el diseño de sistemas distribuidos: la clave de fragmentación que exige el requisito formal de la asignatura (`fecha_apertura`) no coincide con el filtro más frecuente del dominio real del problema (`zone`, usado por el control de acceso por rol de técnico y por la mayoría de los reportes operativos). Se priorizó el cumplimiento literal del requisito sobre la preferencia de diseño original, documentando explícitamente en el ADR-0003 el costo aceptado —consultas por zona convertidas en *scatter-gather* sobre las cuatro particiones— en vez de ocultarlo o de forzar un diseño híbrido no solicitado. En un sistema de producción real, esta decisión probablemente se resolvería con una fragmentación compuesta (por ejemplo, subparticiones por zona dentro de cada partición temporal) o con un índice secundario global cubriendo sobre `zone`; ambas alternativas quedan fuera del alcance de esta entrega pero se identifican como extensión natural del diseño actual.

8.2. Sobre la colocación cliente–ticket entre microservicios

La limitación más significativa frente a la letra de la guía es la imposibilidad de colocar físicamente `tickets` con la tabla de clientes, porque esta última es propiedad de `auth-service` en una base de datos distinta. Esta tensión —requisito de la guía de Entrega 3 (pensada para un esquema monolítico o de bases de datos compartidas) contra un principio central de la arquitectura de microservicios adoptada desde la Entrega 2 (cada servicio es dueño exclusivo de su propio esquema)— se resolvió documentando la limitación en vez de comprometer el límite de propiedad de datos entre servicios, que es una de las garantías arquitectónicas más valiosas de la descomposición en microservicios [6]: permite evolucionar o escalar `ticket-service` sin coordinar cambios de esquema con `auth-service`. La alternativa de aplicación —resolución de identidad del cliente vía JWT en vez de una clave física compartida— preserva esta garantía a costa de no poder ejecutar un *join* físico dentro del motor de base de datos entre ambas tablas.

8.3. Sobre la instrumentación y la observabilidad

La combinación de pruebas de integración con Testcontainers y métricas Prometheus verificadas con `promtool` (Sección 5) resultó más valiosa de lo previsto inicialmente para diagnosticar el propio incidente de saturación de memoria descrito en esa sección: sin las métricas de reintentos y sin comprender el comportamiento esperado del aislamiento serializable —verificado previamente en las pruebas de integración—, habría sido más difícil distinguir un problema de recursos del sistema operativo (memoria insuficiente) de un problema de diseño en la capa de datos. Esto refuerza un principio general de sistemas distribuidos: la observabilidad no es solo un requisito de cumplimiento de la rúbrica, sino una herramienta de diagnóstico que se vuelve indispensable precisamente cuando el sistema falla de forma inesperada.

8.4. Sobre la representatividad del pipeline analítico

El pipeline de Spark descrito en la Sección 6 se diseñó deliberadamente para reflejar un caso de uso analítico propio del dominio (reincidencia y *clustering* de incidencias) en vez de aplicar las cinco transformaciones exigidas de forma genérica sobre datos sin relación con el negocio. Esto tiene un costo: los resultados de *speedup* del protocolo experimental (Sección 7) son específicos a un pipeline dominado por una etapa de *join*/ventana con *shuffle* moderado y una etapa final de aprendizaje

automático relativamente costosa (vectorización TF-IDF + KMeans), y no necesariamente generalizan a pipelines de Spark dominados por *shuffles* masivos entre tablas de tamaño comparable. Se documenta esta amenaza a la validez externa explícitamente en la Sección 7 en vez de presentar la conclusión del experimento como universalmente aplicable a cualquier carga de trabajo de Spark.

8.5. Alcance no cubierto en esta entrega

Dos elementos mencionados en la guía de la asignatura —un *API Gateway* explícito como punto de entrada único al conjunto de microservicios, y la orquestación completa de los cinco microservicios mediante un único archivo `docker-compose.yml` raíz— no se implementaron en esta entrega y quedan identificados como extensión pendiente: actualmente cada microservicio se levanta y orquesta de forma independiente, sin una capa de enrutamiento y políticas transversales (limitación de tasa, agregación de respuestas) centralizada.

9. Conclusiones e integración con E4

Este trabajo abordó dos dimensiones de la Entrega 3 sobre la base de código de microservicios construida en entregas anteriores: la fragmentación horizontal correcta y verificable de la capa de datos, y el procesamiento analítico paralelo de un volumen de datos que excede lo razonable para un solo proceso. Sobre la primera dimensión, se rediseñó la tabla `tickets` de un esquema fragmentado por zona geográfica a uno fragmentado por rango de `fecha_apertura`, verificando formalmente las tres condiciones de corrección de toda fragmentación horizontal —completitud, reconstrucción y disyunción— y documentando explícitamente, en vez de ocultar, los *trade-offs* aceptados: degradación de las consultas por zona a *scatter-gather* sobre cuatro particiones, y la imposibilidad de colocalización física con la tabla de clientes por pertenecer a un microservicio distinto. La corrección del comportamiento transaccional del clúster —en particular, el rechazo explícito de escrituras concurrentes conflictivas propio del aislamiento `SERIALIZABLE`— se verificó empíricamente contra una instancia real de CockroachDB mediante Testcontainers, no mediante inspección manual ni simulación, y se instrumentó con métricas Prometheus validadas sin advertencias por el *linter* oficial de la herramienta.

Sobre la segunda dimensión, se implementó un pipeline de Apache Spark con las cinco categorías de transformación exigidas, aplicadas de forma coherente a un objetivo analítico propio del dominio —reincidencia de clientes y agrupamiento de incidencias por texto y zona— y ejecutado de extremo a extremo sobre un conjunto sintético de 600,000 registros con resultados reproducibles y no triviales: 68.3% de reincidencia detectada dentro de una ventana de 30 días y una distribución de *clusters* no degenerada. Siguiendo metodología estándar de experimentación en ingeniería de software, se ejecutó además un protocolo formal de 50 corridas contrastando este pipeline contra un baseline secuencial en pandas, con un resultado que no era el esperado y que precisamente por eso es valioso: para este volumen de datos, el baseline secuencial resultó significativamente más rápido que Spark incluso en su configuración de mayor paralelismo ($p \approx 5 \times 10^{-10}$), aunque Spark sí exhibe un *speedup* interno real al escalar de uno a ocho núcleos (ajuste de Amdahl $p \approx 0,53$). Reportar este resultado tal como ocurrió, en vez de solo buscar el resultado que confirmara la hipótesis inicial, es coherente con el principio de honestidad aplicado en todo este documento.

El único resultado numérico que depende de una ejecución prolongada y de coordinación logística del equipo que sigue sin ejecutarse es la caracterización de tolerancia a fallos ante la caída de uno y dos nodos: se documenta en este trabajo con su diseño e implementación completos, pero su valor numérico final se reporta como pendiente de la ejecución programada para el cierre de esta entrega,

en línea con el compromiso de no sustituir mediciones reales por estimaciones a lo largo de todo el documento.

9.1. Integración con la Entrega 4

Siguiendo el principio de complementariedad E3→E4 de la guía de esta entrega —todo artefacto producido en E3 debe reutilizarse, ampliarse o instrumentarse en E4, no descartarse—, los siguientes artefactos quedan disponibles para consumo directo en el siguiente ciclo:

- **El esquema de base de datos fragmentado** (`db-cluster/scripts/init_db.sql`, ADR-0003) y el propio **clúster de tres nodos de CockroachDB** quedan como la capa de persistencia definitiva del sistema: E4 no necesita rediseñar ni migrar datos, solo construir sobre ella la interfaz web, la interfaz móvil y la observabilidad exigidas en el siguiente ciclo.
- **Las tres métricas Prometheus** instrumentadas en la Sección 5 (`crdb_query_duration_seconds`, `crdb_transaction_retries_total`, `crdb_pool_active_connections`) se emplean tal cual, sin modificación, como fuente de datos del panel de observabilidad consolidado que exige E4 —por eso se instrumentaron en esta entrega y no se aplazaron.
- **Las pruebas de integración con Testcontainers** quedan incorporadas a la suite de pruebas del proyecto como regresión permanente: cualquier cambio futuro sobre `ticket-service` en E4 se sigue verificando contra una instancia real de CockroachDB, no contra una base embebida.
- **El pipeline de Spark y el protocolo experimental** (`spark/src/pipeline.py`, `baseline.py`, `protocol_experiment.py`) quedan como la base analítica reutilizable: E4 puede extenderlos con nuevas transformaciones o exponer sus resultados (reincidencia, *clusters* de incidencias) a través de la interfaz de usuario, sin rehacer la integración JDBC ni el diseño experimental ya validado aquí.

Como trabajo futuro más allá de lo directamente heredable por E4, se identifican dos líneas adicionales: una estrategia de fragmentación compuesta que preserve el beneficio de poda de particiones tanto para consultas temporales como para consultas por zona, y la validación del pipeline analítico sobre un conjunto de datos real de incidencias de un ISP, para contrastar la representatividad del dataset sintético usado en esta entrega frente a la distribución real de texto y de cardinalidad de clientes del dominio.

Referencias

- [1] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*. Springer, 3rd ed., 2011.
- [2] R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley, 1991.
- [3] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [4] M. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” Tech. Rep. RFC 7519, IETF, 2015.

- [5] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2nd ed., 2021.
- [7] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference (AFIPS '67)*, pp. 483–485, ACM, 1967.
- [8] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pp. 305–319, USENIX Association, 2014.
- [9] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, "CockroachDB: The resilient geo-distributed SQL database," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pp. 1493–1509, ACM, 2020.
- [10] E. Brewer, "CAP twelve years later: How the rules have changed," *IEEE Computer*, vol. 45, no. 2, pp. 23–29, 2012.
- [11] D. Abadi, "Consistency tradeoffs in modern distributed database system design: CAP is only part of the story," *IEEE Computer*, vol. 45, no. 2, pp. 37–42, 2012.
- [12] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pp. 15–28, USENIX Association, 2012.
- [13] S. Salloum, R. Dautov, X. Chen, P. X. Peng, and J. Z. Huang, "Big data analytics on apache spark," *International Journal of Data Science and Analytics*, vol. 1, no. 3–4, pp. 145–164, 2016.
- [14] B. Chambers and M. Zaharia, *Spark: The Definitive Guide: Big Data Processing Made Simple*. O'Reilly Media, 2018.
- [15] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC '10)*, pp. 143–154, ACM, 2010.
- [16] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [17] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.
- [18] J. L. Gustafson, "Reevaluating amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
- [19] Cockroach Labs, "CockroachDB documentation." <https://www.cockroachlabs.com/docs/>, 2024. Accedido: julio 2026.